

Lesson 7 – Practice with Functions

You've learned what a function is — a reusable block of code that performs one task.

Now you'll get to *create your own* and see how they make your programs more powerful and organised.

Exercise 1: Your First Function

Goal: Write and call your own function that performs a task.

Instructions:

1. Create a function called `start_message()` that prints a short welcome message.
2. Call the function **twice** so the message appears two times.
3. Add a creative touch: maybe a divider line or emoji.

Example structure (fill in your own lines):

```
def start_message():  
    # Your code here  
  
# Call the function twice  
start_message()  
start_message()
```

Question:

Why is it better to call a function twice than to copy and paste the same code twice?

Exercise 2: Add Customisation with Parameters

Goal: Make a function that can greet *any* person.

Instructions:

1. Create a function called `greet(name)`.
2. Inside the function, print a message like: `Hello, <name>!`
3. Call your function three times with different names.

Starter code:

```
def greet(name):  
    # Your code here  
  
greet("Jim")  
greet("Ava")  
greet("Liam")
```

Question:

How does using a parameter make your function more flexible?

Exercise 3: Add Decision-Making

Goal: Combine what you know about **if statements** with functions.

Instructions:

1. Write a function called `check_age(age)`.
2. If the age is 18 or more, print `"You can enter!"`.
3. Otherwise, print `"Sorry, you're too young!"`.
4. Ask the user for their age and call your function with their answer.

Hint: Convert the input into an integer using `int()`.

Starter code:

```
def check_age(age):  
    # Your code here  
  
user_age = int(input("Enter your age: "))  
check_age(user_age)
```

Challenge:

Add a check so that if the user enters a negative number, the program prints `"That's not a valid age!"`.

Exercise 4: Functions That *Return* Values

New concept introduction:

So far, your functions have **printed** messages.

But functions can also **return** information — this means they *send a result back* to the program that called them.

Here's the difference:

```
# This prints the answer inside the function  
def say_hi():  
    print("Hi!")  
  
# This sends the answer back to be used elsewhere  
def double(number):  
    return number * 2  
  
result = double(4)  
print(result) # prints 8
```

You can also have multiple parameters (more than one piece of data).

Example:

```
def add(a, b):  
    return a + b
```

Your Task:

1. Create a function called `area_rectangle(width, height)` that **returns** (not prints) the area of a rectangle.
2. Call the function and store the result in a variable called `result`.
3. Print out the result in a sentence like: `"The area is 15 square units."`
4. Try calling your function several times with different numbers.

Starter code:

```
def area_rectangle(width, height):  
    # calculate the area  
    # return the result  
  
# Call the function  
result = area_rectangle(5, 3)  
print("The area is", result, "square units.")
```

Questions:

1. What does the word **return** mean in a function?
2. How is `return` different from `print()`?
3. Why might you want your function to return a value instead of printing it?

Exercise 5: Mini Project – Function Toolkit

Goal: Use what you've learned to build something of your own!

Task:

Create **at least three functions** that help the user with small tasks.

Ideas:

- `greet_user(name)` – greets someone by name
- `check_even(number)` – tells if a number is even or odd
- `convert_cm_to_meters(cm)` – converts centimetres to metres

Bonus Challenge:

Create a “*main menu*” where the user picks which tool to use.

Tip: You'll need to use `input()` and a few function calls.

Reflection

Answer these in 1–2 sentences each:

1. What is one advantage of using functions in your programs?
2. What's the main difference between `print()` and `return`?
3. Which exercise did you find most enjoyable, and why?